

Cahier des Charges – Chiffrement par Blocs (SPN)

1. Présentation du projet:

Dans le cadre de ce projet, on propose de concevoir et d'implémenter une interface graphique pour un algorithme de chiffrement par blocs basé sur un Réseau de Substitution-Permutation (SPN). Cette interface assurera le chiffrement et le déchiffrement des messages à l'aide d'une clé secrète partagée, ainsi que l'analyse de sa sécurité et le test de sa résistance aux attaques par force brute .

2. Objectifs:

- Implémenter un algorithme de chiffrement SPN fonctionnel from scratch .
 - Analyser la sécurité de l'algorithme en testant l'effet d'avalanche .
 - tester la résistance du chiffrement en lançant une attaque de “brute force” .
 - Introduire une interface graphique capable de :
 - chiffrer un message donné;
 - déchiffrer un message donné;
 - analyser la sécurité de l'algorithme;
 - lancer une attaque;
-

3. Spécifications techniques:

- **Langage de programmation:** python 3.8
- **IDE utilisée:** Visual Studio Code
- **Bibliothèque graphique:** Tkinter

- **Architecture:** Réseau de Substitution-Permutation
 - **Taille du bloc:** 8 bits
 - **Taille de la clé principale:** 16 bits
 - **Nombre de sous-clés:** 5 sous-clés
 - **Taille des sous-clés:** 8 bits
-

4. Architecture retenue:

▪ **S-box:**

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
A	3	B	F	6	2	7	4	5	C	D	E	0	1	9	8

Justification: ce choix des valeurs assure:

- La bijectivité
- La non- linéarité
- La confusion (relation complexe entre le message chiffré et la clé)

▪ **Table de permutation:**

0	1	2	3	4	5	6	7
3	0	4	6	7	1	2	5

Justification: ce choix des valeurs assure:

- La diffusion
- L'augmentation de l'effet Avalanche

▪ **Key schedule:**

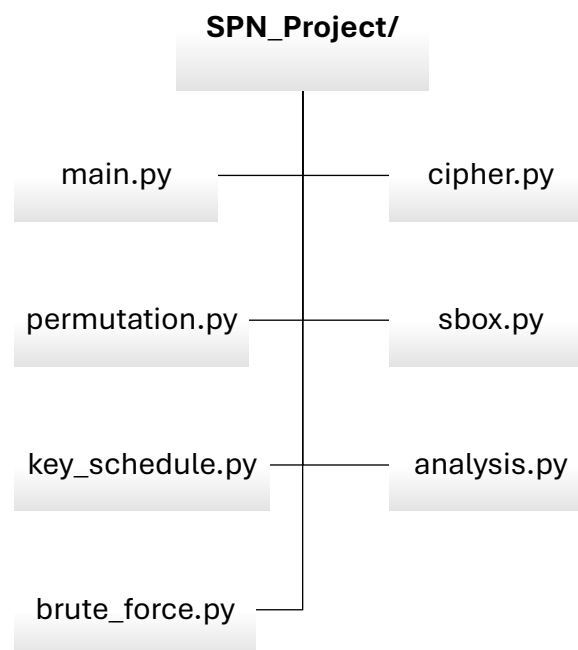
On va générer 5 sous-clés de 8 bits à partir d'une clé principale de 16 bits. La génération des sous-clés se fait de la manière suivante:

- ✓ **1^{re} sous-clé:** les 8 premiers bits de la clé principale.
- ✓ **2^e sous-clé:** les 8 derniers bits de la clé principale.
- ✓ **3^e sous-clé:** on applique une rotation circulaire gauche de 3 bits sur la clé principale, puis extraire les 8 bits de poids fort.

- ✓ **4^e sous-clé:** on applique une rotation circulaire gauche de 5 bits sur la clé principale, puis extraire les 8 bits de poids fort.
 - ✓ **5^e sous-clé:** on applique une rotation circulaire gauche de 7 bits sur la clé principale, puis extraire les 8 bits de poids fort.
-

5. Fonctionnalités:

6. Organisation logicielle:



-
- ✓ **main.py:** Fichier principal du projet et lancement de l'interface graphique.
 - ✓ **cipher.py:** Implémente les fonctions de chiffrement et de déchiffrement.

- ✓ **permutation.py**: Gère la permutation et la permutation inverse.
 - ✓ **sbox.py**: Gère la S-Box et la S-Box inverse.
 - ✓ **key_schedule.py**: Génère les sous-clés à partir de la clé principale.
 - ✓ **analysis.py**: Analyse l'effet avalanche et génère les statistiques.
 - ✓ **brute_force.py**: Implémente l'attaque par force brute.
-

7. Planning:

Expected duration

Se	

8. Tests de validation:

Le projet doit assurer les propriétés suivantes:

- La Réversibilité: le message chiffré doit pouvoir être déchiffré à l'aide de la même clé afin de retrouver le message original.
- Le Déterminisme: le chiffrement d'un même message avec une même clé doit toujours donner la même message chiffrée.
- La Sensibilité clé: le chiffrement d'un message avec deux clés différentes doit donner deux messages chiffrés différents.
- L'Effet Avalanche: la modification d'un bit doit entraîner un changement de 50% des bits du message chiffré.